

Python Automation

Complete Notes: Beginner to Advanced / Expert

Python automation means writing scripts that perform repetitive tasks—such as organizing files, scraping websites, calling APIs, updating Excel, sending emails, or testing web apps—with minimal human intervention. It saves time, reduces manual errors, and scales well across Windows, macOS, and Linux.

1. Setup and Foundations

Install Python 3, then create an isolated environment for each project:

Terminal setup

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate

pip install requests beautifulsoup4 pandas openpyxl
python-dotenv
```

Use a virtual environment to keep each project's dependencies separate, and store secrets such as API keys in environment variables rather than in source code.

Core Python you must know

- Variables and data types: str, int, float, bool, None
- Collections: list, tuple, set, dict
- Conditions: if, elif, else
- Loops: for, while
- Functions: def, parameters, return values
- Modules and packages: import
- File handling: open(), with
- Exception handling: try, except, finally
- OOP basics: classes, objects, methods

Safe file reading

```
def process_file(path):
    try:
        with open(path, "r", encoding="utf-8") as f:
```

```

return f.read()
except FileNotFoundError:
    print(f"File not found: {path}")
    return ""

```

⚠ Exception handling is essential because files may be missing, networks may fail, and websites may change. Use specific exceptions rather than catching every error blindly.

● 2. File and Folder Automation

Python's built-in `os`, `shutil`, and `pathlib` modules are commonly used to create, rename, move, copy, and delete files and directories.

Organize downloads by file type

```

from pathlib import Path
import shutil

source = Path("downloads")
target = Path("organized")

for file in source.iterdir():
    if file.is_file():
        folder = target / file.suffix.replace(".", "").lower()
        folder.mkdir(parents=True, exist_ok=True)
        shutil.move(str(file), str(folder / file.name))
    print(f"Moved: {file.name} -> {folder}")

```

Task	Module / method
List files	<code>Path.iterdir()</code> , <code>os.listdir()</code>
Create folder	<code>Path.mkdir()</code> , <code>os.makedirs()</code>
Rename / move	<code>shutil.move()</code>
Copy	<code>shutil.copy()</code> , <code>shutil.copy2()</code>
Delete file	<code>Path.unlink()</code>
Delete folder	<code>shutil.rmtree()</code>
Check existence	<code>Path.exists()</code>

⚠ Always test file-deletion or file-move scripts on a dummy folder first, because a bad script can damage real data.

● 3. CSV, Excel, and PDF Automation

Use csv for simple spreadsheets, pandas for data cleaning and analysis, and openpyxl for creating or styling Excel files.

Create an Excel summary

```
import pandas as pd

df = pd.read_csv("sales.csv")
df["Total"] = df["Quantity"] * df["Price"]
summary = df.groupby("Region")["Total"].sum().reset_index()
summary.to_excel("sales_summary.xlsx", index=False)
```

Common data-automation tasks

- Read and clean CSV/Excel data
- Remove duplicates and missing values
- Merge multiple files
- Generate reports
- Apply formulas and formatting
- Export results to Excel, CSV, or a database

Clean an Excel report

```
df = pd.read_excel("report.xlsx")
df = df.drop_duplicates()
df = df.dropna(subset=["Customer"])
df.to_csv("clean_report.csv", index=False)
```

● 4. Web Scraping

Web scraping extracts information from web pages. A typical approach uses requests to fetch HTML and BeautifulSoup to parse it.

Basic web scraper

```
import requests
from bs4 import BeautifulSoup
```

```
url = "https://example.com"
response = requests.get(url, timeout=10)
response.raise_for_status()

soup = BeautifulSoup(response.text, "html.parser")

for heading in soup.select("h2"):
    print(heading.get_text(strip=True))
```

Scraping workflow

- Inspect the page structure in the browser.
- Fetch the page with requests.
- Parse HTML with BeautifulSoup.
- Select elements using CSS selectors.
- Clean and save the data.
- Add delays, retries, and error handling.

Scrape multiple pages safely

```
import time

for page in range(1, 4):
    url = f"https://example.com/page/{page}"
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    # parse response.text here
    time.sleep(2)
```

⚠ Respect a website's terms of service, robots policy, rate limits, and privacy rules before scraping.

● 5. API Automation

APIs allow scripts to send and receive structured data, usually JSON. The requests library supports GET, POST, PUT, and DELETE operations.

Call an authenticated API

```
import requests

url = "https://api.example.com/items"
```

```
headers = {"Authorization": "Bearer YOUR_API_KEY"}

response = requests.get(url, headers=headers, timeout=10)
response.raise_for_status()

data = response.json()
for item in data.get("results", []):
    print(item["name"])
```

API best practices

- Store API keys in .env files or environment variables.
- Use timeout to prevent scripts from hanging.
- Check response.status_code.
- Handle rate limits and retries.
- Validate JSON before using it.

Load secrets safely

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("API_KEY")
```

● 6. Email and Notification Automation

Python can automate email reports, reminders, and alerts using smtplib, commonly alongside the email package for message formatting.

Send an automated email

```
import smtplib
from email.message import EmailMessage
import os

msg = EmailMessage()
msg["Subject"] = "Daily Report"
msg["From"] = os.getenv("EMAIL_USER")
msg["To"] = "receiver@example.com"
```

```
msg.set_content("Your automated report is ready.")

with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
    server.login(os.getenv("EMAIL_USER"),
os.getenv("EMAIL_PASSWORD"))
    server.send_message(msg)
```

⚠ Use app passwords or OAuth-based authentication where available, and never hard-code credentials in your script.

● 7. Browser Automation

For websites that require login, clicking, form filling, JavaScript rendering, or end-to-end testing, use browser automation tools such as Selenium or Playwright.

Automate a browser

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    page = browser.new_page()
    page.goto("https://example.com")
    page.fill("#search", "Python automation")
    page.click("text=Search")
    page.screenshot(path="result.png")
    browser.close()
```

Browser automation use cases

- Automated form filling
- Login workflows
- Web application testing
- Downloading reports
- Taking screenshots
- End-to-end QA testing

□ For testing, organize checks with pytest and use assertions to verify expected behavior.

● 8. GUI and Desktop Automation

GUI automation controls the mouse, keyboard, windows, and desktop applications. pyautogui is a common library for this purpose.

Control keyboard input

```
import pyautogui
import time

time.sleep(3)
pyautogui.typewrite("Automated text", interval=0.05)
pyautogui.press("enter")
```

⚠ Use GUI automation only when no API, CLI, or file-based method is available, because it can be fragile if screen resolution, window position, or application layout changes.

● 9. Scheduling Automation

To run scripts automatically, use Windows Task Scheduler, cron on macOS/Linux, the Python schedule library, or cloud schedulers such as GitHub Actions, AWS Lambda, and Azure Functions.

Run a daily job

```
import schedule
import time

def job():
    print("Running daily report...")

schedule.every().day.at("09:00").do(job)

while True:
    schedule.run_pending()
    time.sleep(60)
```

□ On Linux/macOS, cron can run a Python script at a fixed time; on Windows, Task Scheduler performs the same role.

● 10. Logging, Error Handling, and Reliability

Production automation should log what it does, recover gracefully from expected failures, and fail clearly when it cannot continue.

Add logging and error handling

```
import logging

logging.basicConfig(
    filename="automation.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

try:
    logging.info("Task started")
    # automation logic
except Exception as e:
    logging.exception("Task failed")
    raise
finally:
    logging.info("Task finished")
```

Reliability checklist

- Use specific try/except blocks.
- Add retries for network operations.
- Log successes, warnings, and failures.
- Validate inputs before processing.
- Use configuration files or environment variables.
- Test with sample data before using real data.
- Keep scripts modular and reusable.

● 11. Advanced Automation Concepts

Parallel processing

Use `concurrent.futures` when tasks can run independently, such as downloading many files or processing multiple pages.

Run tasks in parallel

```
from concurrent.futures import ThreadPoolExecutor
import requests

urls = ["https://example.com/1", "https://example.com/2"]

def fetch(url):
    response = requests.get(url, timeout=10)
    return response.status_code

with ThreadPoolExecutor(max_workers=5) as executor:
    results = executor.map(fetch, urls)

print(list(results))
```

Project structure

Recommended project layout

```
automation_project/
├── .env
├── requirements.txt
├── main.py
├── config.py
├── logger.py
├── modules/
│   ├── files.py
│   ├── excel.py
│   ├── emailer.py
│   └── scraper.py
├── data/
├── output/
└── logs/
```

□ Modular code is easier to test, debug, reuse, and maintain; for example, a single `send_email()` function can be imported by multiple automation scripts.

● 12. Practice Projects

Level	Projects
Beginner	Organize downloads by file type; rename files; merge CSV files; create a daily Excel summary; send a reminder email
Intermediate	Scrape product prices; call a weather API and email a forecast; download files on a schedule; clean sales data; automate browser form filling
Advanced	End-to-end web-testing suite; API → cleaning → database → report → email pipeline; cloud backups; multi-threaded scraper; reusable automation framework

● 13. Learning Roadmap

Level	Focus	Outcome
Beginner	Python basics, files, CSV, simple scripts	Automate one repetitive task
Intermediate	APIs, pandas, Excel, web scraping, email	Build useful workflow automations
Advanced	Browser automation, scheduling, logging, parallelism, testing	Build reliable production-grade systems

● 14. Database Automation

Databases automate data storage, retrieval, updates, and reporting. Start with SQLite because it needs no server, then move to MySQL, PostgreSQL, or MongoDB.

SQLite database automation

```
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS employees (
id INTEGER PRIMARY KEY,
name TEXT,
department TEXT,
salary REAL
)
""")

cursor.execute(
"INSERT INTO employees (name, department, salary) VALUES (?, ?, ?)",
("Aman", "IT", 55000)
)

conn.commit()

for row in cursor.execute("SELECT * FROM employees"):
    print(row)

conn.close()
```

SQL automation tasks

- Insert records from CSV/Excel
- Update records based on business rules
- Generate daily reports

- Back up tables
- Migrate data between systems
- Schedule database cleanup jobs

Export a database table to Excel

```
from sqlalchemy import create_engine
import pandas as pd

engine = create_engine("sqlite:///company.db")
df = pd.read_sql("SELECT * FROM employees", engine)
df.to_excel("employees.xlsx", index=False)
```

● 15. Cloud Automation

Cloud automation lets Python manage servers, storage, databases, and deployments through SDKs and APIs.

Upload a file to AWS S3

```
import boto3

s3 = boto3.client("s3")

s3.upload_file(
    "report.pdf",
    "my-automation-bucket",
    "reports/report.pdf"
)

print("File uploaded")
```

Cloud	Python use cases
AWS	S3 uploads, EC2 control, Lambda, DynamoDB
Google Cloud	Cloud Storage, BigQuery, Cloud Functions
Azure	Blob Storage, Azure Functions, Logic Apps
GitHub	Repository automation, Actions, issue bots

⚠ Never expose cloud access keys in code. Use IAM roles, environment variables, or a secrets manager.

● 16. CI/CD Automation

CI/CD automatically tests, builds, and deploys code whenever developers push changes. GitHub Actions is widely used for Python automation and browser-test pipelines.

GitHub Actions workflow

Create `.github/workflows/automation.yml`:

.github/workflows/automation.yml

```
name: Python Automation Tests

on:
  push:
  branches: [main]
  pull_request:
  branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          playwright install --with-deps
```

```
- name: Run tests
run: pytest --tracing=retain-on-failure

- name: Upload test artifacts
if: always()
uses: actions/upload-artifact@v4
with:
name: test-results
path: test-results/
```

□ This workflow clones the repository, installs Python and dependencies, installs Playwright browsers, runs pytest, and uploads failure traces as artifacts.

● 17. Testing Automation with pytest

pytest is the standard Python testing framework for unit tests, API tests, and browser automation tests. It supports fixtures, parameterization, markers, plugins, and parallel execution.

Basic pytest tests

```
# test_math.py
from calculator import add

def test_add():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-2, -3) == -5
```

Run tests

```
pytest -v
```

Fixtures

Reusable test data

```
import pytest
```

```
@pytest.fixture
def sample_data():
    return {"name": "Aman", "age": 25}

def test_user(sample_data):
    assert sample_data["name"] == "Aman"
```

Parameterized tests

Test multiple inputs

```
import pytest

@pytest.mark.parametrize("a,b,expected", [
    (2, 3, 5),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

● 18. Advanced Browser Automation with Playwright

Playwright supports Chromium, Firefox, and WebKit, and is designed for reliable end-to-end web testing with auto-waiting, assertions, tracing, and parallel execution.

Install Playwright

```
pip install pytest-playwright playwright
playwright install
```

Browser automation script

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://example.com")
```

```
assert page.title() == "Example Domain"
page.screenshot(path="homepage.png")

browser.close()
```

Pytest + Playwright test

```
from playwright.sync_api import Page, expect

def test_homepage(page: Page):
    page.goto("https://example.com")
    expect(page).to_have_title("Example Domain")
```

Run browser tests

```
pytest
pytest --headed
pytest --browser firefox
```

Page Object Model

pages/login_page.py

```
# pages/login_page.py
class LoginPage:
    def __init__(self, page):
        self.page = page
        self.username = page.get_by_label("Username")
        self.password = page.get_by_label("Password")
        self.submit = page.get_by_role("button", name="Login")

    def login(self, user, pwd):
        self.username.fill(user)
        self.password.fill(pwd)
        self.submit.click()
```

tests/test_login.py

```
# tests/test_login.py
from pages.login_page import LoginPage
```



```
def test_login(page):
    login_page = LoginPage(page)
    login_page.login("demo", "password")
```

□ The Page Object Model keeps page elements and actions separate from test logic, making tests easier to maintain when the website UI changes.

● 19. API Testing Automation

API testing verifies whether backend endpoints return correct status codes, data, and responses. Playwright also provides an API request context for REST API testing.

API test with requests

```
import requests

def test_get_users():
    response =
    requests.get("https://jsonplaceholder.typicode.com/users")
    assert response.status_code == 200

    users = response.json()
    assert len(users) > 0
    assert "name" in users[0]
```

Check	Examples
Status code	200, 201, 400, 401, 404
Performance	Response time and timeout behavior
Data	JSON structure, required fields, pagination
Security	Authentication, authorization, error handling
Negative cases	Invalid input, missing fields, expired tokens

● 20. Data Pipeline Automation

A data pipeline moves data through stages: extract, transform, load, validate, report, and notify.

Pipeline stages

API / Website / Database

↓

Extract data

↓

Clean and transform

↓

Load into database

↓

Generate report

↓

Send email / dashboard update

Simple API-to-database pipeline

```
import pandas as pd
import requests
from sqlalchemy import create_engine

url = "https://api.example.com/sales"
data = requests.get(url, timeout=10).json()

df = pd.DataFrame(data)
df = df.drop_duplicates()
df["revenue"] = df["quantity"] * df["price"]

engine = create_engine("sqlite:///sales.db")
df.to_sql("sales", engine, if_exists="replace", index=False)
```

Tool	Best for
Cron / Task Scheduler	Simple scheduled scripts
schedule	Python-managed scheduling
Apache Airflow	Complex data pipelines
Prefect	Modern Python workflow orchestration
GitHub Actions	Code testing and deployment
Docker	Consistent deployment environments

● 21. AI and Chatbot Automation

AI automation uses language-model APIs to summarize documents, classify messages, generate reports, answer questions, or process unstructured text.

AI-powered text summarization

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Summarize customer feedback briefly."},
        {"role": "user", "content": "The product is good but delivery was slow."}
    ]
)

print(response.choices[0].message.content)
```

AI automation use cases

- Email classification and auto-replies
- Resume screening
- Customer-support chatbots
- Document summarization
- Data extraction from PDFs
- Report generation
- Code-review assistance

⚠ Always validate AI output before using it for critical business decisions.

● 22. Security and Secrets Management

Automation scripts often handle passwords, API keys, database credentials, and personal data. Secure handling is mandatory.

Use environment variables

```
import os
from dotenv import load_dotenv

load_dotenv()

db_password = os.getenv("DB_PASSWORD")
api_key = os.getenv("API_KEY")
```

Security rules

- Never commit .env, passwords, tokens, or keys to GitHub.
- Add .env to .gitignore.
- Use least-privilege access for cloud services.
- Rotate API keys regularly.
- Encrypt sensitive files where required.
- Validate all external input.
- Use HTTPS for API calls.
- Log errors without exposing secrets.

● 23. Docker for Automation

Docker packages your script, dependencies, and environment into one portable container.

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

Build and run the container

```
docker build -t python-automation .
docker run --env-file .env python-automation
```

□ Docker is useful when your automation must run identically on your laptop, a server, and CI/CD.

● 24. Logging, Monitoring, and Alerts

For reliable automation, monitor whether jobs run successfully and alert the right person when they fail.

Monitoring and alerting

```
import logging
import requests

logging.basicConfig(
    filename="job.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

def send_alert(message):
    logging.error(message)
    # Send Slack, email, or Teams alert here

try:
    logging.info("Automation started")
    # main automation task
except Exception as error:
    send_alert(f"Automation failed: {error}")
```

Monitoring checklist

- Log start time, end time, and duration
- Log record counts processed
- Capture errors with stack traces
- Send alerts for failures
- Store logs centrally
- Retry transient network failures
- Track success rate over time

● 25. Expert-Level Project Portfolio

Project	Skills demonstrated
Automated file organizer	pathlib, shutil, scheduling
Excel reporting bot	pandas, openpyxl, email automation
Price tracker	requests, BeautifulSoup, database, alerts
Job scraper	Web scraping, CSV/Excel export, scheduling
Weather alert system	API automation, email/Slack notifications
Social media post scheduler	APIs, authentication, cron
Invoice processor	PDF extraction, Excel, database
E-commerce test suite	Playwright, pytest, Page Object Model
CI/CD test pipeline	GitHub Actions, Docker, test reports
Cloud backup system	Boto3, logging, retries, monitoring
AI email assistant	OpenAI API, classification, email automation
End-to-end data pipeline	API → cleaning → database → dashboard → email

● 26. Final Expert Roadmap

Stage	Duration	What to learn
Beginner	Weeks 1-4	Python basics, files, CSV, loops, functions
Intermediate	Weeks 5-10	APIs, pandas, Excel, scraping, email
Advanced	Weeks 11-16	Playwright, pytest, databases, scheduling
Professional	Weeks 17-24	CI/CD, Docker, cloud, monitoring
Expert	Ongoing	Airflow, scalable pipelines, AI automation, system design

□ A strong automation engineer does not only write scripts—they build reliable, tested, secure, scheduled, and maintainable systems.

● At a Glance

Area	Main tools	Typical output
Files & folders	pathlib, os, shutil	Organized, renamed, backed-up files
Data & Excel	pandas, openpyxl, csv	Clean reports and dashboards
Web & APIs	requests, BeautifulSoup, Playwright	Extracted data and automated workflows
Email & alerts	smtplib, email, Slack APIs	Automated notifications
Testing	pytest, Playwright	Reliable automated test suites
Scheduling & CI/CD	cron, schedule, GitHub Actions	Hands-free recurring execution
Cloud & DevOps	Boto3, Docker, cloud SDKs	Scalable deployed automation
AI automation	OpenAI API, LangChain	Smart document and message workflows